Lab Exercise 5 & 6: Domain-Based React Application

Problem Statement: In Part 1 Lab Exercises (Exercise 1,2,3 and 4), you developed a module of your selected domain using HTML, CSS, and JavaScript. In Part 2, you are required to implement the different modules using React by applying React concepts such as components, state management, event handling and reusable UI design. The application should further extend the existing module by implementing Create, Read, Update, and Delete (CRUD) operations using a suitable data storage mechanism.

Mandatory Requirements:
Reuse the same domain and module developed in Part 1 and implement the following using React:

- Functional Components
- JSX
- useState() Hook
- Event Handling
- Form Handling
- Multiple Input Fields
- Radio Buttons
- Select Dropdown
- Reusable Components
- CSS Styling
- CRUD Operations (Create – Add a new record, Read – Display all records, Update – Modify an existing record, Delete – Remove a record)
- Bonus: Search operation to filter records dynamically

Database Options:

Option 1: Local Storage (Beginner)	**Option 2: JSON Server (Recommended)**	**SQLite / MongoDB + Express (Advanced)**
No installation required Suitable for beginners Stores data in the browser	Simulates a REST API Supports HTTP methods (GET, POST, PUT, DELETE) Recommended for React CRUD applications	Suitable for students familiar with Node.js and Express Recommended for full-stack application development

Expected Enhancements:

Compared to your HTML/CSS/JavaScript implementation, the React version should demonstrate:

- Component-based architecture
- State management using React Hooks
- Reusable UI components
- Improved code organization and maintainability
- Dynamic rendering using React
- Database integration with CRUD functionality

Deliverables

- React source code
- Database files (db.json, SQLite database, or Local Storage implementation)
- Screenshots demonstrating:
    - Create
    - Read
    - Update
    - Delete
    - (Bonus) Search
- Brief report (1–2 pages)
- Project demonstration (5–7 minutes)

Note: Students should develop the application with responsive design, it can later be converted into a mobile application using one of these approaches

Progressive Web App (PWA)	**Capacitor (Recommended)**	**React Native (Advanced)**
Runs like a mobile app. Installable from the browser. No changes to React code except PWA configuration. Works offline with service workers.	Wraps the React web app as an Android/iOS application. Students can generate an APK and install it on their phones. Minimal code changes.	Requires rewriting the UI using React Native components. Best suited for a separate mobile development course.